

FORTRAN 90

Algunas cosas básicas del lenguaje

Fortran 90 se usan las siguientes formas para hacer declaraciones de variables, donde `"type-specifier "` es uno de las siguientes "keywords": `INTEGER`, `REAL`, `LOGICAL`, `COMPLEX` y `CHARACTER`, y después una `lista` de identificadores o variables separados por comas.

`type-specifier :: list`

Ejemplos:

```
INTEGER :: Zip, Total, counter
REAL    :: AVERAGE, x, Difference
LOGICAL :: Condition, OK
COMPLEX :: Conjugate
```

Las variables tipo "character" requieren información adicional: *string length*

En general el tipo `CHARACTER` deberá estar seguido por el atributo que le dice a la variable de que tamaño máximo guardará (`LEN = l`), donde *l* es la longitud del "string".

- la "`LEN=`" es opcional.
- Si la longitud del string es uno se puede usar `CHARACTER` sin el atributo `LEN`.

Ejemplos:

```
CHARACTER(LEN=20) :: Answer, Quote
```

Las variables `Answer` y `Quote` pueden guardar cadenas de hasta 20 caracteres.

`CHARACTER(20) :: Answer, Quote` es lo mismo que arriba.

`CHARACTER :: Keypress` significa que la variable `Keypress` puede guardar **UN** solo carácter (*i.e.*, length 1).

El atributo `PARAMETER`

Un identificador `PARAMETER` es un atributo que se da aparte del tipo a aquellas variables que no van a ser modificadas a lo largo del programa. En otras palabras, estas son *constantes*.

El atributo `PARAMETER` se usa después del tipo en la declaración de la variable, cada variable puede ser seguida por un `=` y después seguida por su valor.

```
INTEGER, PARAMETER :: MAXIMUM = 10
REAL, PARAMETER :: PI = 3.1415926, E = 2.17828
LOGICAL, PARAMETER :: TRUE = .true., FALSE = .false.
```

En Fortran 90 las cadenas son de longitud *fija*, debemos recordar lo siguiente:
Si un “string” es mas largo que lo que se encuentra en la longitud de **PARAMETER**, lo de la derecha será truncado.

Si el “string” es mas corto que la longitud de **PARAMETER**, espacios serán agregados a la derecha.

```
CHARACTER (LEN=4) , PARAMETER :: ABC = "abcdef"
CHARACTER (LEN=4) , PARAMETER :: XYZ = "xy"
```

Se guardarán: ABC = a b c d

Y en XYZ = x y _ _ donde _ indican espacios en blanco.

Operadores Aritméticos

Hay cuatro tipos de operadores en Fortran 90: Aritmético, relacional, lógico y carácter. Lo siguiente muestra los primeros tres tipos de operaciones:

<i>Tipo</i>	<i>Operador</i>	<i>Asociatividad</i>
Aritmético	**	<i>Derecha a izquierda</i>
	* /	Izquierda a derecha
	+ -	Izquierda a derecha
Relacional	< <= > == /=	Ninguno
Lógicos	.NOT.	<i>Derecha a izquierda</i>
	.AND.	Izquierda a derecha
	.OR.	Izquierda a derecha
	.EQV. .NEQ.	Izquierda a derecha

La precedencia de las operaciones es: ** Tiene la mas alta; * y / son las siguientes, seguidos por + y -. Todos los operadores relacionales después al final los 5 operadores relacionales donde .EQV. y .NEQV. son los de más baja precedencia.

Evaluación de expresiones

“Las expresiones aritméticas son evaluadas de izquierda a derecha”. Una expresión “*single mode*” es aquella donde los operandos son del mismo tipo. Ejemplo si todos los operandos son enteros, la respuesta es entera. Si los operandos tienes distintos tipos este es “*mixed mode*”.

INTEGER y **REAL** construye **REAL**, y el operando **INTEGER** es convertido a **REAL** antes de la evaluación. Ejemplo: 3.5*4 es convertido a 3.5*4.0 pasando a ser “single mode”. Hay una excepción: **x**INTEGER**: **x**3** es **x*x*x** y **x**(-3)** es 1.0/(**x*x*x**). **x**REAL** es evaluado con **log()** y **exp()**.

La instrucción de asignación

La instrucción de asignación tiene la forma:

`variable = expression`

Si el tipo de `variable` y `expression` son idénticos, el resultado es guardado en `variable`. Si el tipo de `variable` y `expression` son de distinto tipo, el resultado de `expression` es convertido al tipo de `variable`. Si `expression` es `REAL` y `variable` es `INTEGER`, el resultado es truncado.

En el siguiente ejemplo se usa la inicialización de variable `Unit`, y el uso correcto de `PARAMETER PI`.

```
INTEGER :: Total, Amount
INTEGER :: Unit = 5
Amount = 100.99
Total = Unit * Amount
```

Otro ejemplo:

```
REAL, PARAMETER :: PI = 3.1415926
REAL :: Area
INTEGER :: Radius
Radius = 5
Area = (Radius ** 2) * PI
```

Está ultima expression es equivalente a: `Radius ** 2 * PI`

Funciones intrínsecas en Fortran

Fortran provee muchas y variadas funciones que son muy comunes, y uno se refiere a ellas como: *funciones intrínsecas*. Algunas muy usadas en matemáticas son:

<i>Función</i>	<i>Significado</i>	<i>Tipo arg.</i>	<i>Tipo regresado</i>
<code>ABS (x)</code>	absoluto valor de x	<code>Entero</code>	<code>Entero</code>
		<code>Real</code>	<code>Real</code>
<code>SQRT (x)</code>	Raíz cuadrada de x	<code>Real</code>	<code>Real</code>
<code>SIN (x)</code>	Seno de x en radianes	<code>Real</code>	<code>Real</code>
<code>COS (x)</code>	Coseno de x en radianes	<code>Real</code>	<code>Real</code>
<code>TAN (x)</code>	Tangente de x en radianes	<code>Real</code>	<code>Real</code>
<code>ASIN (x)</code>	Arc seno de x	<code>Real</code>	<code>Real</code>
<code>ACOS (x)</code>	Arc coseno de x	<code>Real</code>	<code>Real</code>
<code>ATAN (x)</code>	Arc tan de x	<code>Real</code>	<code>Real</code>
<code>EXP (x)</code>	Exponencial de x	<code>Real</code>	<code>Real</code>
<code>LOG (x)</code>	Logaritmo natural de x	<code>Real</code>	<code>Real</code>
<code>INT (x)</code>	Trunca la parte decimal de x	<code>Real</code>	<code>Entero</code>
<code>NINT (x)</code>	Redondea al entero más próximo	<code>Real</code>	<code>Entero</code>

FLOOR (x)	El entero mas grande menor o igual que x	Real	Entero
FRACTION (x)	La parte fraccionaria de x	Real	Real
REAL (x)	Convierte a x en real	Entero	Real
MAX (x1, x2, ..., xn)	Obtiene el máximo de x1,x2...	Entero	Entero
		Real	Real
MIN (x1, x2, ..., xn)	Obtiene el mínimo de x1,x2..	Entero	Entero
		Real	Real
MOD (x,y)	remainder x - INT (x/y) *y	Entero	Entero
		Real	Real

Ejemplos:

```

INT (-3.5) → -3
NINT (3.5) → 4
NINT (-3.4) → -3
FLOOR (3.6) → 3
FLOOR (-3.5) → -4
FRACTION (12.3) → 0.3
REAL (-10) → -10.0

```

¿Qué es **IMPLICIT NONE**?

Fortran tiene una tradición interesante: Todas las variables que empiezan en su nombre con **i, j, k, l, m** y **n**, Si no son declaradas, son del tipo **INTEGER** por default. Y si el nombre empieza en el conjunto **(a-h)** o en el conjunto **(o-z)** la variable se considera de tipo real. Esta propiedad puede causar serias consecuencias cuando se programa si no se tiene el debido cuidado. **IMPLICIT NONE** significa que todas las variables tienen que ser declaradas, no hay declaración implícita. Todos los programas en esta clase deberán ser de tipo **IMPLICIT NONE**. ¡Puntos menos serán dados a tu calificación si no los usas!

Instrucción **READ**

En Fortran 90 se usa la instrucción **READ (*,*)** para leer datos en variables del teclado en tiempo de ejecución:

```
READ (*,*) v1, v2, ..., vn
```

```
READ (*,*)
```

La segunda forma tiene un significado especial.

Ejemplo:

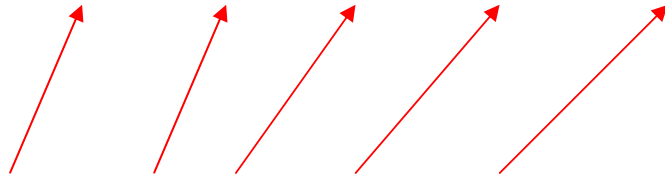
```

INTEGER :: Age
REAL :: Amount, Rate
CHARACTER (LEN=10) :: Name
READ (*,*) Name, Age, Rate, Amount

```

READ (*,*) lee datos del teclado por defecto, sin embargo uno puede usar en unix el direccionador para leer de un archivo. Los datos leídos estarán separados por espacios podrán separados por líneas múltiples. *La ejecución de READ (*,*) siempre empieza con una línea nueva !*. Entonces, esta instrucción leerá cada constante en la correspondiente variable.

```
CHARACTER(LEN=5) :: Name
REAL :: height, length
INTEGER :: count, MaxLength
READ(*,*) Name, height, count, length, MaxLength
```

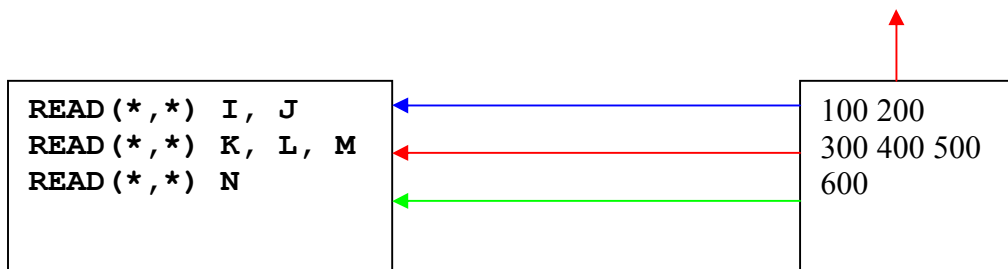


Input: "Smith" 100.0 25 123.579 10000

Hay que tener cuando los datos se leen de distintas líneas:

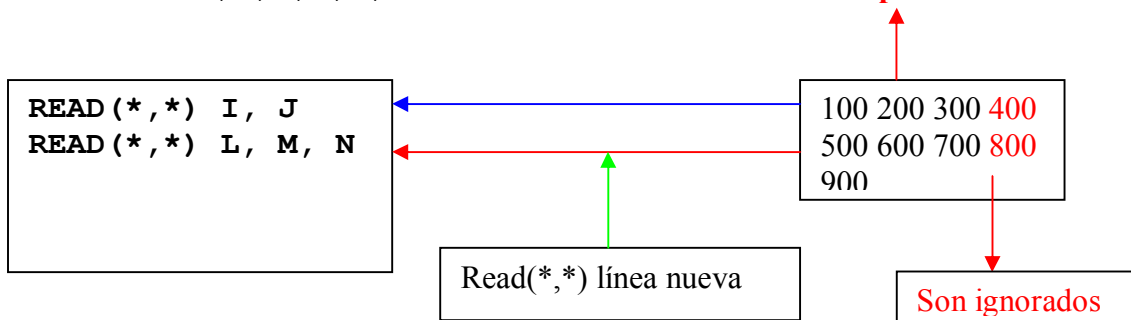
```
INTEGER :: I, J, K, L, M, N
```

Input:



```
INTEGER :: I, J, K, L, M, N
```

Input:



La instrucción **READ (*,*)** siempre empieza con una línea nueva, **READ (*,*)** sin ninguna variable significa salta a la línea siguiente!

```
INTEGER :: P,Q,R,S
```

```
READ(*,*) P,Q
READ(*,*)
READ(*,*) R,S
```

```
100 200 300
300 400 500
700 800 900
```

Ignorados

Instrucción WRITE

En Fortran 90 se usa la instrucción `WRITE(*,*)` para escribir información a la pantalla. `WRITE(*,*)` tiene dos formas, donde `exp1,exp2, ..., expn` son expresiones se tiene:

`WRITE(*,*) exp1, exp2, ..., expn`

`WRITE(*,*)`

`WRITE(*,*)` calcula el resultado da cada expresión e imprime en pantalla.

`WRITE(*,*)` *siempre empieza con una línea nueva!*

Aquí tenemos un ejemplo simple:

```
INTEGER :: Target
REAL :: Angle, Distance
CHARACTER(LEN=*) , PARAMETER ::
Time = "The time to hit target ", &
IS = " is ", &
UNIT = " sec."
Target = 10
Angle = 20.0
Distance = 1350.0
WRITE(*,*) 'Angle = ', Angle
WRITE(*,*) 'Distance = ', Distance
WRITE(*,*) _____
WRITE(*,*) Time, Target, IS, &
Angle * Distance, UNIT
```

No se conoce la longitud

Símbolos de continuacion

Línea en blanco

Salida:

```
Angle = 20.0
Distance = 1350.0
The time to hit target 10 is 27000.0 sec.
```

Ejemplo completo:

Este programa calcula la posición (coordenadas x y y) y la velocidad (magnitud y dirección) de un proyectil, dado el tiempo t , la velocidad de lanzamiento, u , y el ángulo inicial de lanzamiento, a (grados), y $g=9.8$ será la aceleración de la gravedad. Los desplazamientos horizontales y verticales, x y y , son calculados como sigue:

$$x = u \cdot \cos(a) \cdot t$$
$$y = u \cdot \sin(a) \cdot t - g \cdot t^2 / 2$$

Los componentes horizontales y verticales de la velocidad son calculados como

$$V_x = u \cdot \cos(a)$$

$$V_y = u \cdot \sin(a)$$

La magnitud del vector velocidad es:

$$V = (V_x^2 + V_y^2)^{1/2}$$

El ángulo del vector velocidad es :

$$\tan(a) = V_y / V_x$$

El siguiente programa lee el ángulo de lanzamiento a , el tiempo desde el lanzamiento t , y la velocidad del lanzamiento u , y se calcula con esto la posición, la velocidad y el ángulo.

```
PROGRAM Projectile
IMPLICIT NONE
REAL, PARAMETER :: g = 9.8 ! acceleration due to gravity
REAL, PARAMETER :: PI = 3.1415926
REAL :: Angle ! launch angle in degree
REAL :: Time ! time to flight
REAL :: Theta ! direction at time in degree
REAL :: U ! launch velocity
REAL :: V ! resultant velocity
REAL :: Vx ! horizontal velocity
REAL :: Vy ! vertical velocity
REAL :: X ! horizontal displacement
REAL :: Y ! vertical displacement
READ(*,*) Angle, Time, U
Angle = Angle * PI / 180.0 ! convert to radian
X = U * COS(Angle) * Time
Y = U * SIN(Angle) * Time - g*Time*Time / 2.0
Vx = U * COS(Angle)
Vy = U * SIN(Angle) - g * Time
V = SQRT(Vx*Vx + Vy*Vy)
Theta = ATAN(Vy/Vx) * 180.0 / PI ! convert to degree
WRITE(*,*) 'Horizontal displacement : ', X
WRITE(*,*) 'Vertical displacement : ', Y
WRITE(*,*) 'Resultant velocity : ', V
WRITE(*,*) 'Direction (in degree) : ', Theta
END PROGRAM Projectile
```

Operador //

Fortran 90 usa `//` para concatenar dos strings. Si el strings **A** y **B** tienen longitudes *m* y *n*, la concatenación `A // B` es un string de longitud *m+n*.

Ejemplo de que sucede cuando concatenamos:

```
CHARACTER(LEN=4) :: John = "John", Sam = "Sam"
CHARACTER(LEN=6) :: Lori = "Lori", Reagan = "Reagan"
CHARACTER(LEN=10) :: Ans1, Ans2, Ans3, Ans4
Ans1 = John // Lori ! Ans1 = "JohnLori_"
Ans2 = Sam // Reagan ! Ans2 = "Sam_Reagan"
Ans3 = Reagan // Sam ! Ans3 = "ReaganSam_"
Ans4 = Lori // Sam ! Ans4 = "Lori__Sam_"
```

Tipo "carácter" : el Substring

Una parte o porción de un "string" o cadena de caracteres consecutivos es un *substring*. Para seleccionar un substring, es posible agregar un *extent specifier* a la variable de tipo `CHARACTER`. Un "extent specifier" tiene la siguiente forma:

`( integer-exp1 : integerexp2 )`

La primera y la segunda expresión indican el inicio y el fin del substring: `(3:8)` significa de carácter 3 al 8, En la expresión `(integer-exp1:integer-exp2)`, si el primer `exp1` falta, el substring empieza en el primer carácter y si `exp2` falta, el substring termina en el último carácter.

Ejemplo:

Si `A = "12345678"`, entonces `A(:5)` es `"12345"`

Y si `A(3+x:)` is `"5678"` where `x` is 2.

Los substrings pueden ser usados del algún lado del operador de asignación.

Supongamos que tenemos el siguiente substring:

`LeftHand = "123456789"`

(su longitud es 10) .

Ahora tendríamos :

`LeftHand(3:5) = "abc"` construye `LeftHand = "12abc67890"`

`LeftHand(4:) = "lmnopqr"` construye `LeftHand = "123lmnopqr"`

`LeftHand(3:8) = "abc"` construye `LeftHand = "12abc__90"`

`LeftHand(4:7) = "lmnopq"` construye `LeftHand = "123lmno890"`

¿Qué es el atributo KIND?

Fortran 90 tiene `KIND` un atributo que sirve para seleccionar la precisión de una variable/constante numerica. El `KIND` de una constante o variable es un numero positivo entero (o más como veremos más adelante) y puede ser tratada como una constante.

Ejemplo:

`126_3 : 126` es un entero que tiene un `KIND 3`

`3.1415926_8` : `3.1415926` es un real of `KIND 8`

La función intrínseca `SELECTED_INT_KIND(k)` selecciona el `KIND` de un entero, donde el valor de *k*, es un entero positivo que significa el entero que tiene un valor entre -10^{**k} y 10^{**k} . Así, el valor de *k* es aproximadamente el número de dígitos de ese `KIND`. Por ejemplo, `SELECTED_INT_KIND(10)` significa un entero `KIND` de no más de 10 dígitos. Si `SELECTED_INT_KIND()` regresa `-1`, esto significa que el hardware no soporta el pedido dado a `KIND`.

El `SELECTED_INT_KIND()` es usualmente usado para la especificación del tipo de variable como se muestra a continuación:

```
INTEGER, PARAMETER :: SHORT = SELECTED_INT_KIND(2)
INTEGER(KIND=SHORT) :: x, y
```

El ejemplo de arriba muestra como se declara un entero (`INTEGER`) con el atributo `PARAMETER` y `SHORT` igual a `SELECTED_INT_KIND(2)`, el cual es `KIND` de 2-dígitos enteros. Entonces el atributo `KIND=` especifica que las variables `x` y `y` son de tipo `INTEGER` y solo pueden contener valores con dos dígitos. En el programa, es posible usar la notación `-12_SHORT` y `9_SHORT` para escribir constantes de ese `KIND`.

¿Qué es `KIND REAL`?

Se usa `SELECTED_REAL_KIND(k,e)` para especificar `KIND` de una variable/constante `REAL`, donde *k* es el numero de cifras significantes y *e* es el numero de dígitos en el exponente, *k* y *e* deberán ser numeros positivos. Notar que *e* es opcional.

`SELECTED_REAL_KIND(7,3)` selecciona un `REAL KIND` de 7 cifras significativas y 3 dígitos para el exponente: $\pm 0.xxxxxxx \cdot 10^{\pm yyy}$

Ejemplo:

```
INTEGER, PARAMETER :: &
SINGLE=SELECTED_REAL_KIND(7,2), &
DOUBLE=SELECTED_REAL_KIND(15,3)
REAL(KIND=SINGLE) :: x
REAL(KIND=DOUBLE) :: Sum
x = 123.45E-5_SINGLE
Sum = Sum + 12345.67890_DOUBLE
```

¿Porqué `KIND`?

En el Fortran77 con el uso de `INTEGER*2`, `REAL*8`, `DOUBLE PRECISION`, etc se especificaba la “precisión” de una variable. Por ejemplo, `REAL*8` significaba el uso de 8 bytes para guardar un valor real. Esto resulto no ser muy portable ya que algunas computadoras por ejemplo no pueden usar 2 bytes para un “short integer” (i.e., `INTEGER*2`). Además el uso de `kind` nos permite tener mas control en la precisión. Debido a las diferencias entre computadoras ya sea en su hardware o en su arquitectura, debemos tener cuidado:

Cuando hacemos una petición de **KIND** puede que no sea posible. Por ejemplo, **SELECTED_INT_KIND(100)** Puede que no sea realista para la mayoría de las computadoras.

Los compiladores encontrarán la mejor manera para una petición de **KIND**. Si un valor **KIND** es “grande” y este es guardado en una variable con menor **KIND** pueden ocurrir comportamientos o valores inesperados. Se deberá usar **KIND** con cuidado para una máxima portabilidad.